Bench wiring sheet - protocol emulator on the MachXO3L kit (3 Nov 2026)

One wire per row: **FROM** one end **TO** the other, **AND** a third end where the wire has one (a pull-up, a second board). Kit pins are written **J4-n (ball)**: n is the numbered hole on header J4, the ball name (K12...) is the FPGA's own label printed beside it, the same connection under a second name, for checking by continuity. **Go by the hole number**: the ball letters H and M (H16 = hole 25, M16 = hole 10) are easy to misread. **Engine pin** numbers (0-19) are what the chip's programs call that pin. Everything is **3.3 V logic** (FPGA and ESP32 pins are not 5 V tolerant); one common ground between every board.

1. Core: kit to ESP32 host. Fit these eight first, then rerun.ps1 (command r): ID reads 0x0022, 0 problems.

FROM	TO	AND (third end)	What it is
J4-3 (K12)	ESP32 GPIO 5		CS, host SPI
J4-5 (M14)	ESP32 GPIO 23		MOSI, host SPI
J4-7 (L14)	ESP32 GPIO 19		MISO, host SPI
J4-9 (M15)	ESP32 GPIO 18		SCK, host SPI
J4-11 (GND)	ESP32 GND	every other board's GND	common ground
J4-4 (K13)	ESP32 GPIO 4		IRQ: high while any engine has data
J4-6 (N14)	ESP32 GPIO 16		engine pin 0: engine A's UART out, into ESP32 Serial2 RX
J4-26 (G15)	ESP32 GPIO 17		engine pin 15: the ESP32 sends test pulses into the chip (glitch filter)

J4-1 and J4-2 are the kit's 3.3 V (printed IO1, the bank supply, not a ball name): use for pull-ups only, never join them to the ESP32's 3.3 V. Kit: **bash fpga/machxo3/build.sh load** (volatile SRAM, never the NVCM). ESP32 (the scripts find its port themselves): **esp32\upload_and_watch.ps1 -Sketch chip_test**.

2. I2C bus (engine pins 2 = SCL, 3 = SDA). ESP32 as the slave: rerun.ps1 -Cmd i. UNO R3 as the slave: -Cmd n.

FROM	TO	AND (third end)	What it is
J4-10 (M16) SCL	ESP32 GPIO 22 or UNO A5	5.1 k resistor to 3.3 V	SCL, engine pin 2
J4-13 (L15) SDA	ESP32 GPIO 21 or UNO A4	5.1 k resistor to 3.3 V	SDA, engine pin 3
UNO GND	common ground		only when the UNO is the slave; its own 5 V pull-ups are switched off by its sketch
3.3 V for the two pull-ups	J4-1 or the UNO's 3.3 V pin		one source, not both

Never both slaves at once (both answer at address 0x50): unplug the UNO's A4 and A5 for -Cmd i. UNO sketch: arduino/i2c_slave_nano on COM11.

3. SPI master to the 74HC595 (engine pins 4 to 7). rerun.ps1 -Cmd s.

FROM	TO	AND (third end)	What it is
J4-14 (L16)	595 pin 11 SRCLK		SCK, engine pin 4
J4-15 (K14)	595 pin 14 SER		MOSI, engine pin 5
J4-16 (K16)	595 pin 9 QH'		MISO, engine pin 6: the 595's serial out comes back
J4-17 (K15)	595 pin 12 RCLK		CS / latch, engine pin 7
595 pin 16 VCC	3.3 V		power
595 pin 8 GND	common ground		power
595 pin 13 OE	GND		outputs enabled
595 pin 10 SRCLR	3.3 V		not clearing
595 pins 15, 1, 2, 3, 4, 5, 6, 7 (QA..QH)	LED long leg	LED short leg -> 330 ohm -> GND	red, yellow or green only (white/blue stay dim at 3.3 V; the flat-topped ones never lit); the run ends on 0x55 so QA (pin 15) is high

Before the rebuild the LEDs stayed dark with the chip side proven good: meter pins 16, 8, 13, 10 first, then pin 15 after a run, then the QH' lead.

4. Every J4 hole the chip uses (for anything new)

J4 holes	Balls	Engine pins	Direction
6, 8, 10, 13, 14, 15, 16, 17	N14, N16, M16, L15, L16, K14, K16, K15	0, 1, 2, 3, 4, 5, 6, 7	in and out (open drain able)
18, 19, 20	J14, H14, J15	8, 9, 10	out only
23, 24, 25	J16, H15, H16	11, 12, 13	out only
26, 27, 28, 29, 30	G15, G16, F15, F16, E15	15, 16, 17, 18, 19	in only
1, 2	3.3 V		
11, 12	GND		

Holes and balls pair up in order: hole 6 is ball N14 (engine pin 0), hole 8 is N16 (pin 1), and so on along each row. Kit LEDs: P16 lights for IRQ, N15 is the heartbeat (they light when driven low).
Cyclone IV board (black EP4CE6E22C8N core board): everything on the right-hand header, pins 38-72, both bank jumpers at 3.3. CS 38, SCK 39, MOSI 42, MISO 43, IRQ 44 to the same ESP32 GPIOs as section 1; engine pin 0 = 46 (to GPIO 16), pins 1-7 = 49 50 51 52 53 54 55 (I2C SCL 50 / SDA 51; SPI 52-55), pins 8-13 = 58 59 60 64 65 66, pin 15 = 67 (from GPIO 17), 16-19 = 68-71. Clock 50 MHz on pin 24, LEDs 1 (IRQ) and 2 (heartbeat). Needs a USB Blaster on the JTAG header (ordered 3 Nov, due 5-6 Nov): bash fpga/cyclone4/build.sh load. Sketch: CHIP_HZ = 50e6. This board runs three engines (ID 0x0023); the kit two.
Lesson of 4 Nov night: an instrument on the bus is powered or unclipped, never off and attached. The logic analyser switched off with its probes still on SCL and SDA clamped both lines through its input diodes: the EEPROM NAKed its address, writes failed, reads came back FD FD FD FD, while the chip's own sniffer showed the bus faithfully. Powered on, everything passed again.

What is plugged in right now (4 Nov 2026, midday): no double booking

A new program may only use a FREE hole; moving a lead means updating this list. TAKEN: hole 1 (IO1 3.3 V: the keypad's four 10 k pull-ups); holes 3, 4, 5, 7, 9 (the host, ESP32 or Pico: CS, IRQ, MOSI, MISO, SCK); hole 6 (engine pin 0, the serial line: ESP32 GPIO 16 reads the chip's UART out, or with the Pico as host its GP0 sends to the chip for pico/uart_rx.py, the ESP32 lead out of the hole then, 4 Nov night); holes 10 and 13 (I2C SCL/SDA with 5.1 k pull-ups, to the UNO's A5 and A4 plus GND, confirmed 4 Nov by -Cmd n PASS); holes 14-17 (keypad columns, tail 5-8; the 74HC595 is out of circuit since 4 Nov); hole 26 (ESP32 GPIO 17 test pulses, the regression needs it); holes 27-30 (keypad rows, tail 1-4, 10 k to hole 1). hole 19 (engine pin 9, H14: the IR emitter's signal lead since 4 Nov evening); hole 8 (engine pin 1: the KY-022 IR receiver's S lead since 4 Nov afternoon; the HC-SR04 echo needs it later, one at a time); hole 2 (second 3.3 V: the receiver's +). **FREE**: holes 18 and 20 (engine pins 8 and 10, out only; 18 pencilled for the HC-SR04 trigger);

holes 23, 24, 25 (engine pins 11-13, out only). ESP32 pins taken: 4, 5, 16, 17, 18, 19, 23, plus 21 and 22 only if it is the I2C slave; VIN (5 V) is free for the HC-SR04. All two-way engine pins 0-7 are spoken for while the keypad and the IR receiver are on.
If it doesn't answer: esp32 rerun.ps1 -Cmd p (lead probe: a FLOATING pin = lead open or wrong hole; ID 0x0000 with MISO driven = CS/SCK/MOSI not arriving; 0xFFFF = MISO open), -Cmd q (IRQ held high 15 s), -Cmd b (I2C wire check: names the chip pin each I2C lead reaches), -Cmd k (GPIO 22 held low 30 s to meter), -Cmd l (repeating ID reads for the oscilloscope), -Cmd y (keypad: names every key pressed; every key 'pressed' = row pull-ups not reaching J4-27..30), -Cmd e (live level of engine pins 15-19 for 30 s, a line on every change: a row with its 10 k in place reads 1). Writes working but reads all zero = MISO lead's J4 end in a hole held low (11/12 GND, 18-25 driven low). The kit forgets its build on any power blip: reload with bash fpga/machxo3/build.sh load.

Order of work after replugging

1. Section 1 only. Load the kit, upload the sketch, rerun.ps1 (command r): ID 0x0022, 0 problems. 2. Add section 2: rerun.ps1 -Cmd i (ESP32 slave) or -Cmd n (UNO slave). 3. Add section 3: rerun.ps1 -Cmd s. 4. Keypad: section 6 (595 leads out), rerun.ps1 -Cmd y.

6. Keypad (rerun.ps1 -Cmd y), IR receiver and ultrasonic: the Elegoo kit parts (4 Nov 2026)

Keypad tail, keys facing you and the tail at the bottom, pin 1 on the left: pins 1-4 are the **rows** top to bottom, 5-8 the **columns** left to right. The chip pulls one column low at a time and reads the rows, so each row needs a 10 k pull-up. The 595 shares J4-14 to 17: unplug its four leads first.

FROM	TO	AND (third end)	What it is
keypad pin 1 (row 1: 1 2 3 A)	J4-27 (G16)	10 k resistor to J4-1 (3.3 V)	row 1, engine pin 16
keypad pin 2 (row 2: 4 5 6 B)	J4-28 (F15)	10 k resistor to J4-1	row 2, engine pin 17
keypad pin 3 (row 3: 7 8 9 C)	J4-29 (F16)	10 k resistor to J4-1	row 3, engine pin 18
keypad pin 4 (row 4: * 0 # D)	J4-30 (E15)	10 k resistor to J4-1	row 4, engine pin 19
keypad pin 5 (column 1: 1 4 7 *)	J4-14 (L16)		column 1, engine pin 4
keypad pin 6 (column 2: 2 5 8 0)	J4-15 (K14)		column 2, engine pin 5
keypad pin 7 (column 3: 3 6 9 #)	J4-16 (K16)		column 3, engine pin 6
keypad pin 8 (column 4: A B C D)	J4-17 (K15)		column 4, engine pin 7

The sketch names each key and prints the raw map. Every key mirrored (1 reads as *) = the tail is numbered the other way: swap the two groups of four. A key that never shows = its row or column lead.

IR receiver KY-022 (rerun.ps1 -Cmd m: NEC frames as address and command, 'repeat n' while a button is held; -Cmd j: raw pulse lengths) and ultrasonic HC-SR04 (5 V part, echo divided down, program to follow): one at a time, they share J4-8.

FROM	TO	AND (third end)	What it is
IR module S (signal)	J4-8 (N16)		engine pin 1: idles high, pulses low under the 38 kHz bursts
IR module + (VCC)	J4-2 (3.3 V; or J4-1)		power
IR module - (GND)	common ground		power
HC-SR04 VCC	ESP32 5 V pin (VIN)		5 V power
HC-SR04 GND	common ground		power

FROM	TO	AND (third end)	What it is
HC-SR04 Trig	J4-18 (J14)		engine pin 8 (out only): the 10 us trigger; 3.3 V is enough
HC-SR04 Echo	1 k resistor	other end to J4-8 (N16) and a 2 k resistor to GND	echo (5 V) divided to 3.3 V, engine pin 1; 58 us per cm

IR emitter board (rerun.ps1 -Cmd t, loopback to the receiver): signal to J4-19 (H14), G to GND (J4-11 or 12), middle pin not connected. Face the LED at the receiver, a few cm apart.
IR module pads are marked G R Y (ground, power, signal): Y is the S lead. The IR **emitter** (KY-005) must not share J4-8: its LED holds the line low. Pin 1 has a pull-up, so idle-high alone proves nothing about the wiring.
Manhattan remote button codes read on the kit, 4 Nov 2026 (-Cmd m, twice, same both times). NEC timing, 16-bit address and 16-bit command with no inverse, so the sketch prints BAD INVERSE for every key: not a read error. Address word 2916 on every key.

Button	Command word	Button	Command word
1	5CB7	7	5BC7
2	6CB6	8	6BC6
3	7CB5	9	7BC5
4	0CC2	0	3BC9
5	1CC1	POWER	9CB3
6	2CC0	VOL+	2AE0

8. Pico 2 as the host instead of the ESP32 (4 Nov 2026)

One host at a time: move the five SPI/IRQ leads and the ground from the ESP32 to the Pico (kit ends stay put). MicroPython v1.29.0. Files in pico/: chip_host.py, programs.py, id_test.py; copy with pico\put.ps1, run with pico un.ps1 -Script id_test.py. First test: ID 0x0022 five times.

FROM	TO	AND (third end)	What it is
J4-3 (K12)	Pico GP17 (pin 22)		CS
J4-5 (M14)	Pico GP19 (pin 25)		MOSI (SPI0 TX)
J4-7 (L14)	Pico GP16 (pin 21)		MISO (SPI0 RX)
J4-9 (M15)	Pico GP18 (pin 24)		SCK
J4-4 (K13)	Pico GP20 (pin 26)		IRQ
J4-11 (GND)	Pico GND (pin 23)		common ground, between GP18 and GP19

9. AT24C256 EEPROM module on the I2C bus (4 Nov 2026, from the Pico)

Replaces the UNO test slave on J4-10 and J4-13 (the UNO also answers at 0x50: not both). Its pull-ups go to VCC: **power it from 3.3 V only**. Jumpers A0-A2 to GND = address 0x50, WP to GND = writes allowed. Test: pico/run.ps1 -Script at24c.py. **Ground:** the board's negative rail must be wired to a kit GND hole (J4-11 or 12); for a while only the logic analyser's lead tied it in.

FROM	TO	AND (third end)	What it is
module VCC	J4-2 (3.3 V)		power; the D1 LED lights
module SCL	J4-10 (M16)		engine pin 2
module SDA	J4-13 (L15)		engine pin 3
module GND	negative rail	rail wired to J4-11 or 12	ground